

Optimizers & Loss functions.

Optimizers:

An optimizer is an algorithm that updates weights & biases during training thus minimizes the loss functions. In other words, it tells the model how to move to improve predictions.

1.) Gradient Descent:

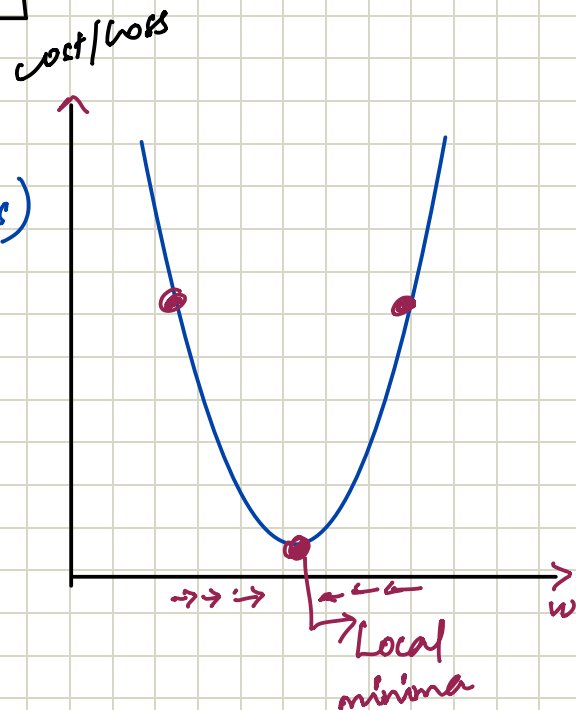
Weight updation formula:

$$W_{\text{new}} = W_{\text{old}} - \eta \frac{\partial L}{\partial W_{\text{old}}}$$

Cons:

* Resource Extensive (Need Huge RAM)

Since all weights are processed at a time.



2.) Stochastic Gradient Descent:

Instead of sending all weights of records, we send one record at a time. So the processing is lil better than Gradient Descent.

Here one record means just one record (or) that represents all in clustered data.

Pros: Ram Usage lesser than G.D

Cons:

- * Very slow convergence.
- * Time Complexity still high

(3.) Mini-Batch SGD:

Instead of one record, batch of (32-256) samples per step.

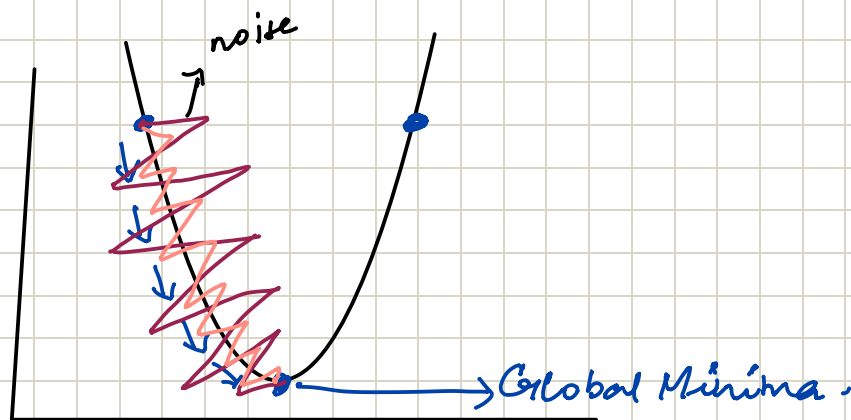
Pros:

Cons:

- * Resource Intensive
- * Convergence (Better)
- * Time complexity (Better)

- * still noise impacts the optimizer.

→ : GD least
→ : SGD High
→ : minibatch SGD Medium



These three are still dealing/impacted by "Noise".
How do we mitigate the noise on optimizers?

"Momentum" $\Rightarrow$ smoothes the journey towards the
(exp moving avg) $\uparrow$ global minima. ($\sum$ to $\sum_2$)

SGD used here is (mini-batch)

(4.) SGD with momentum:

SGD can bounce back & forth like hill but
momentum smoothes it by adding some memory.

Weight Update formula:

$$W_{\text{new}} = W_{\text{old}} - \eta \frac{\partial L}{\partial W_{\text{old}}} \Rightarrow W_t = W_{t-1} - \eta \frac{\partial L}{\partial W_{t-1}}$$

$t \rightarrow \text{timestamp.}$

Exponential Weighted Avg:

t_1	t_2	t_3	...	t_n
a_1	a_2	a_3		a_n

Value $V_{t_1} = a_1$

$$V_{t_2} = \beta * V_{t_1} + (1 - \beta) * a_2$$

$\beta \rightarrow \text{hyperparameter.}$

This helps the smoothening! This concept is applied on

$$\frac{\partial L}{\partial w.}$$

So the formula turns into

$$W_t = W_{t-1} - \eta V_{dw}$$

$$\therefore V_{dw} = \beta * V_{dw_{t-1}} + (1-\beta) \frac{\partial L}{\partial W_{t-1}}$$

Pros:

Quicker Convergence

Noise is minimized

5.) Adagrad (Adaptive Gradient Descent)

Here the learning rate is not fixed as traditionally, the learning rate is adaptive.

Weight Update formula:-

$$W_t = W_{t-1} - \eta \frac{\partial L}{\partial W_{t-1}} \Rightarrow W_t = W_{t-1} - \eta' \frac{\partial L}{\partial W_{t-1}}$$

$$\eta' = \frac{\eta}{\sqrt{\alpha_t + \epsilon}}$$

$$\alpha_t = \sum_{i=1}^t \left(\frac{\partial L}{\partial W_t} \right)^2$$

As time moves, α_t increases, so η' decreases.

Goal: After started training, the Learning rate should be decreasing.

LR decreases towards Global minima.

⊗ Large α_t value $\Rightarrow$ negligible η' change.

⑥ Adadelta and RMSprop:

This optimizer stops the α_t from reaching huge numbers/value.

$$\text{here } \eta' = \frac{\eta}{\sqrt{Sdw + \epsilon}}$$

We would apply exponential weighted average for α_t .

$$Sdw = 0$$

$$Sdw_t = \beta Sdw_{t-1} + (1-\beta) \left(\frac{\partial L}{\partial w_{t-1}} \right)^2$$

Eg: $\beta = 0.95$

$$Sdw_t = (0.95) Sdw_{t-1} + (0.05) \left(\frac{\partial L}{\partial w_{t-1}} \right)^2$$

This 0.05 makes this term smaller.

"Controlled state" by β .

⑦ Adam Optimizer (Momentum + RMSprop).

Smoothing + Adaptive Learning rate

Weight/Bias Update formula:

$$w_t = w_{t-1} - \eta' Vdw$$

$$\eta' = \frac{\eta}{\sqrt{Sdw + \epsilon}}$$

$$Vdw_t = \beta * Vdw_{t-1} + (1-\beta) \frac{\partial L}{\partial w_{t-1}}$$

Momentum: The running avg of past gradients.

RMSprop: The running avg of squared gradients.

Weight Decay:

It's conceptually same as L2 Regularization and add small penalty to the weights to avoid overfitting and generalizes the model well.

Without weight decay:

$$W_{\text{new}} = W_{\text{old}} - \eta \frac{\partial L}{\partial W_{\text{old}}}$$

With weight decay:

$$W = W - \eta \left(\frac{\partial L}{\partial W} + \lambda W \right)$$

λ - too small $\rightarrow$ little effect

λ - Large $\rightarrow$ underfitting

Ⓢ weight decay is regularization technique which prevents overfitting by making network's weights smaller over time.

Ⓢ AdamW (Adam with weight Decay)
(decoupling weight decay from gradient update)

In adam, we add weight decay to the loss function. By doing this, it affects the adaptive Learning rates which hinder optimal convergence (leads to overfitting).

So AdamW is smarter version where it adds weight decay directly to the parameter updates rather than applying on loss function. This gives better regularization & generalization.

Choose Adam: for smaller tasks & regularization is not crucial

Choose AdamW: Larger models or training complex, high dimension data.

Loss functions

Regression:

- * Mean Squared Error
- * Mean Absolute Error
- * Huber loss

Classification

- * Binary Cross Entropy
- * Categorical Cross Entropy

$$\text{Huber Loss} = \begin{cases} \frac{1}{2} (y - \hat{y})^2 & \text{if } |y - \hat{y}| \leq \delta \\ \delta |y - \hat{y}| - \frac{1}{2} \delta^2, & \text{otherwise} \end{cases}$$

Binary Cross Entropy

$$\text{Loss} = -(y) * \log(\hat{y}) - (1-y) * \log(1-\hat{y}) \quad (\text{Log loss})$$

$$\text{Loss} = \begin{cases} -\log(1-\hat{y}) & y=0 \\ -\log(\hat{y}) & y=1 \end{cases}$$

$$\hat{y} = \frac{1}{1+e^{-x}} \quad (\text{Sigmoid})$$

Categorical Cross Entropy (Multiclass Classification)

$$\text{Loss}(x_i, y_i) = - \sum_{j=1}^C y_{ij} * \ln(\hat{y}_{ij})$$

$$\hat{y}_{ij} = \frac{e^{z_{ij}}}{\sum_{j=1}^K e^{z_{ij}}}$$

(Softmax Activation)

